Macronutrient Mass Prediction from Food Images

Senén Marqués

February 5, 2026

1 Aim

The goal of this work is to systematically explore ways in which to predict the macronutrient mass (in grams) from food images. Obtaining SOTA accuracy presents a significant challenge, and in order to address this, our work will be research driven, as we believe that in order to achieve SOTA results, we must make decisions that are anchored in the literature.

2 The Data

We are using the **MM-Food-100K Dataset**, which contains ~100K labeled images of restaurant and home cooked food. Another notable dataset is the **Food101 Dataset**, for classification tasks, which we have seen being used by Mingxing Tan (2020) for transfer learning on ImageNet pretrained CNNs

3 First Steps & A-Priori Knowledge

A priori, we can tackle the problem with two approaches.

The first is to build a pipeline using various pretrained models to: 1) Perform segmentation with classification. 2) Predict volumes for each classified segment. 3) Use the the classifications and volumes to trivially obtain the macronutrient mass. We can use models like **FastSAM** for segmentation, **YOLO26** for segmentation with classification, and others for volume estimation. However, the problem with this approach is that any error from a step can compound over to the next step in the pipeline, and if the individual components are not robust or accurate enough, we cannot expect good results.

The second is to simply train a model with our dataset of labeled images. Regardless of the data and CNN architecture used, this is a much simpler and direct way of confronting the problem and the results depend mainly on the dataset and CNN Architecture used.

Naturally, we opted for the second approach as a starting point.

4 Methodology

4.1 Loading the Data

The first step was to download the *MM-Food-100K Dataset*, to do this, we made a python script to download the dataset csv file and images concurrently and idempotently. Which, after several hours of execution, downloaded the whole 166 GB of images. Then, after having a local copy of the dataset, we made a custom pytorch Dataset class to allow obtaining the image and targets when iterating over it.

4.2 Choosing a CNN Architecture

We have a computer vision task which requires multi output regression, sadly, we noticed that the literature on CNNs is very classification biased. We are aware that we can adapt any CNN for regression by modifying the head of the network and changing the criterion from Cross-Entropy loss to MSE (L2) or MAE (L1) loss. However, almost all the CNNs we reviewed in the literature were benchmarked on ImageNet classification and we don't know which is best suited for multi-output regression. We need to investigate more, but as a starting place, we decided to 1) train from scratch and 2) fine tune an existing model.

In the last decade, the computer vision landscape has been filled with CNN's, however, in recent years,

Vision Transformer have come out and they seem to be the SOTA. However, we are aware that Vision Transformers are computationally expensive and require larger datasets than what we own, so for now we have directed our attention towards SOTA CNNs. The most notable one is **EfficientNet** by Mingxing Tan (2020), which outperforms many other CNNs in accuracy and computation efficiency by degrees of magnitude.

Seeing it as EffcientNet is one of the better CNNs (especially with our 4GB VRAM hardware limitation), we opted to fine tune it and train it from scratch, and observe the results.

Mingxing Tan, Quoc V.Le. 2020. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." *arXiv*.